API Documentation
Book Management API

Backend .NET Internship
Week 3 – Day 5

Prepared by:Mithgal Alhrene
Date: 6-8-2026

1. Introduction

This document describes the Book Management API based solely on the provided Postman Collection and execution screenshots. The API exposes REST endpoints for Create, Read, Update, and Delete (CRUD) operations using JSON payloads.

2. API Overview

Method	Endpoint	Purpose
GET	{{baseUrl}}/api/books	Retrieve all books
GET	{{baseUrl}}/api/books/{id}	Retrieve a book by ID
POST	{{baseUrl}}/api/books	Create a new book
PUT	{{baseUrl}}/api/books/{id}	Update an existing book
DELETE	{{baseUrl}}/api/books/{id}	Delete a book

3. Base URL & Environment

All requests use the Postman Environment variable {{baseUrl}}. Updating this variable allows the same collection to run against different environments without changing request URLs.

4. Endpoints

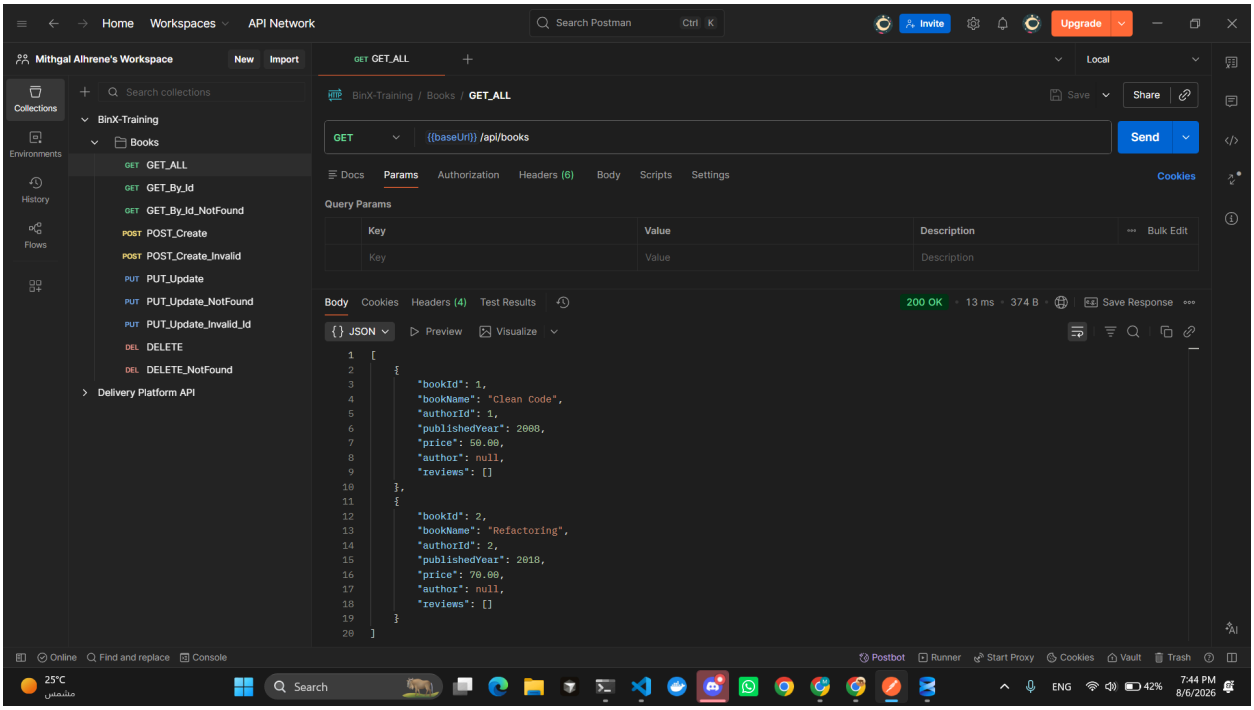
GET_ALL

Method: GET
URL: {{baseUrl}}/api/books
Purpose: Executes the GET ALL operation.

Request Body:

No request body required.

Success Response: 200 OK


Home
Workspaces
API Network
Search Postman
Ctrl K
Invite
Upgrade
Mithgal Alhrene's Workspace
New
Import
Collections
Environments
History
Flows
Search collections
BinX-Training
Books
GET GET_ALL
GET GET_By_Id
GET GET_By_Id_NotFound
POST POST_Create
POST POST_Create_Invalid
PUT PUT_Update
PUT PUT_Update_NotFound
PUT PUT_Update_Invalid_Id
DEL DELETE
DEL DELETE_NotFound
Delivery Platform API
GET GET_ALL
Local
BinX-Training / Books / GET_ALL
Save
Share
GET
{{baseUrl}} /api/books
Send
Docs
Params
Authorization
Headers (6)
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers (4)
Test Results
200 OK
13 ms
374 B
Save Response
JSON
Preview
Visualize
[
{
"bookId": 1,
"bookName": "Clean Code",
"authorId": 1,
"publishedYear": 2008,
"price": 50.00,
"author": null,
"reviews": []
},
{
"bookId": 2,
"bookName": "Refactoring",
"authorId": 2,
"publishedYear": 2018,
"price": 70.00,
"author": null,
"reviews": []
}
]
Online
Find and replace
Console
Postbot
Runner
Start Proxy
Cookies
Vault
Trash
25°C
Search
ENG
42%
7:44 PM
8/6/2026

GET_By_Id

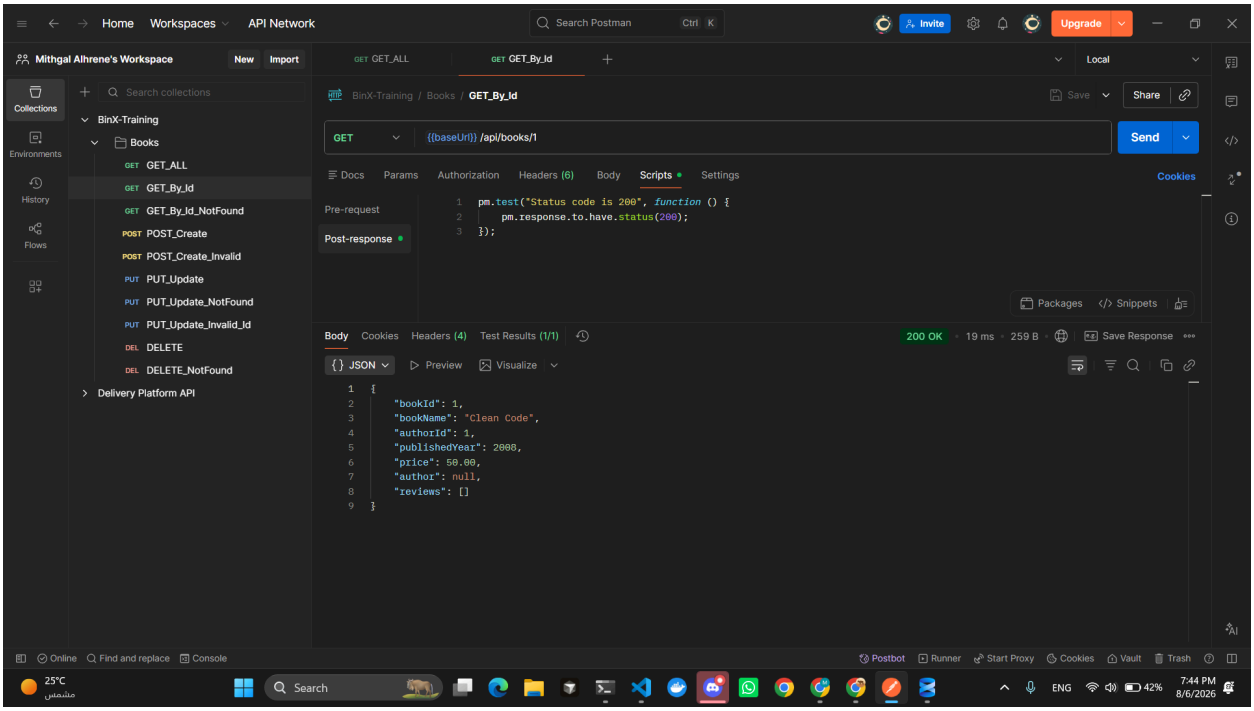
Method: GET
URL: {{baseUrl}}/api/books/1
Purpose: Executes the GET By Id operation.

Request Body:

No request body required.

Success Response: 200 OK

Possible Error Response: 404 Not Found



GET_By_Id_NotFound

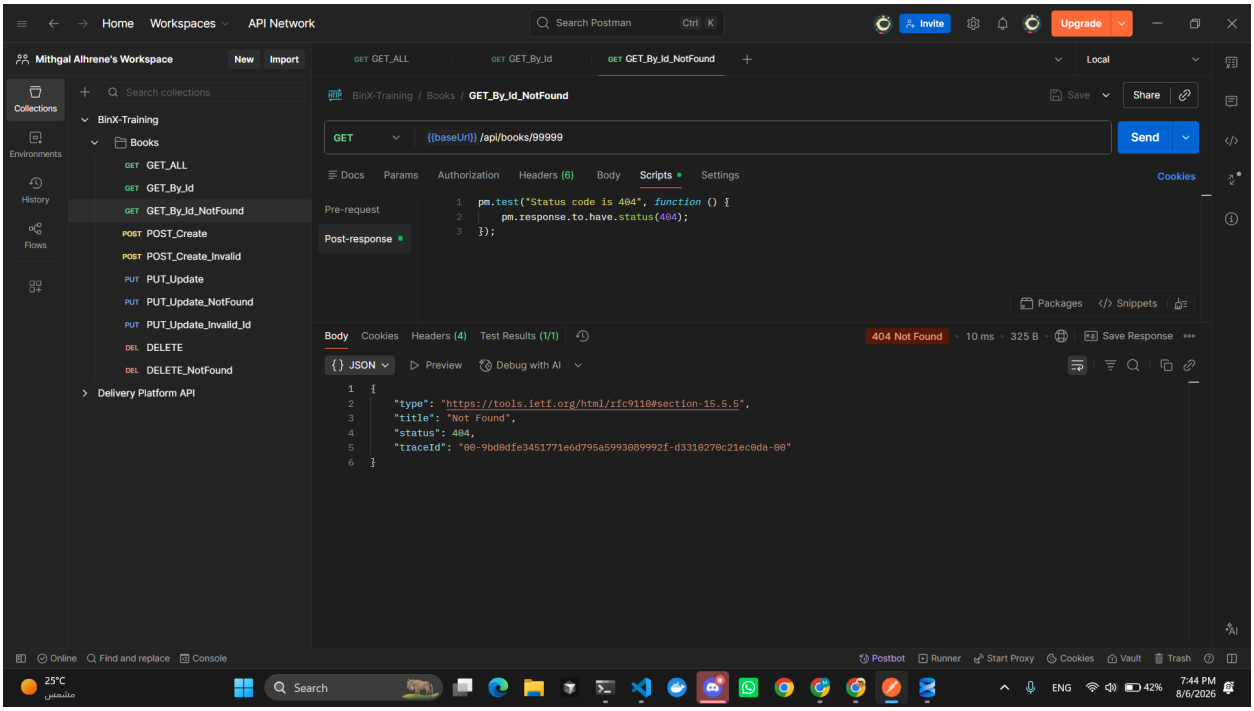
Method: GET
URL: {{baseUrl}}/api/books/99999
Purpose: Executes the GET By Id NotFound operation.

Request Body:

No request body required.

Success Response: 404 Not Found

Possible Error Response: 404 Not Found



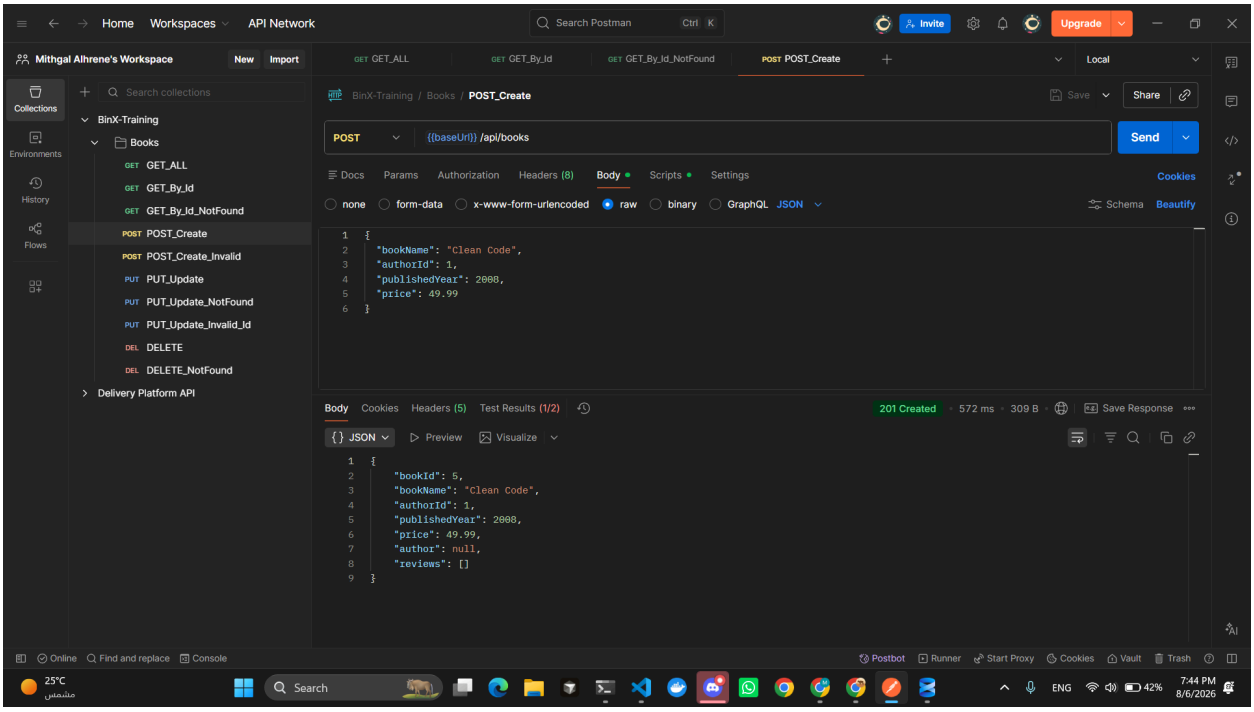
POST_Create

Method: POST
URL: {{baseUrl}}/api/books
Purpose: Executes the POST Create operation.

Request Body:

> ***{ "bookName":"Clean Code","authorId":1,"publishedYear":2008,"price":49.99 }***

Success Response: 201 Created



POST_Create_Invalid

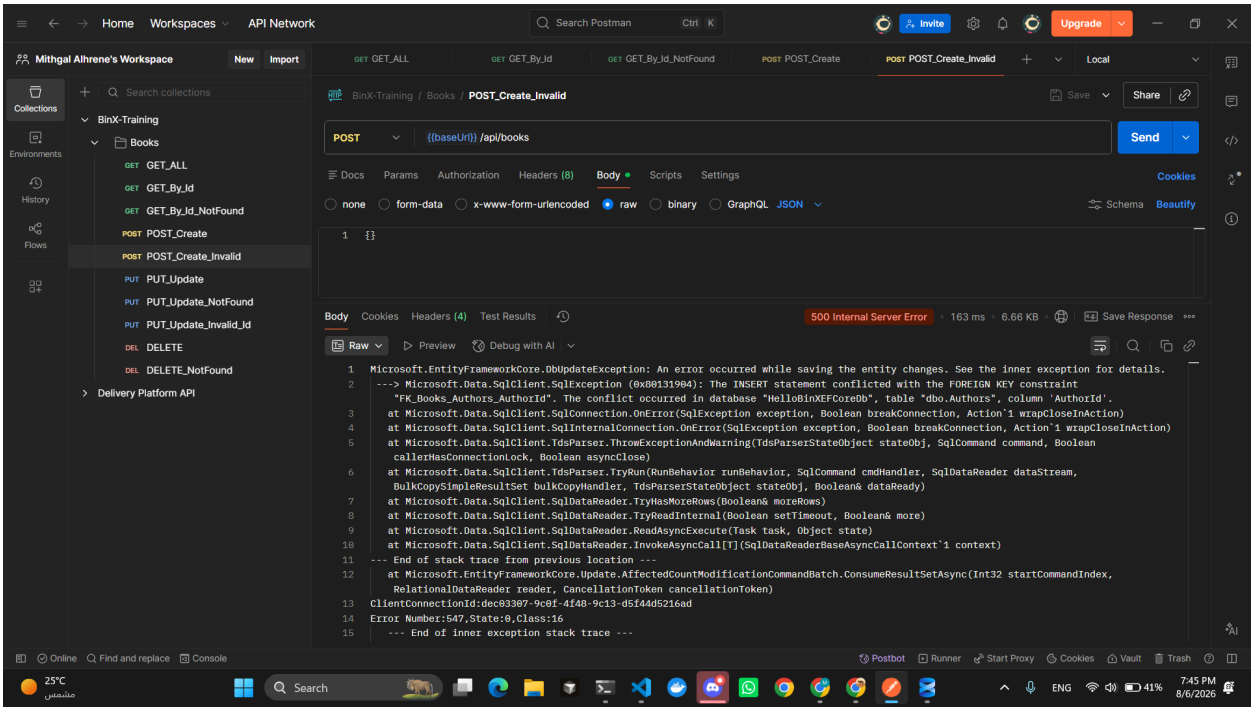
Method: POST
URL: {{baseUrl}}/api/books
Purpose: Executes the POST Create Invalid operation.

Request Body:

```
{}
```

Success Response: 500 Internal Server Error

Possible Error Response: 500 Internal Server Error



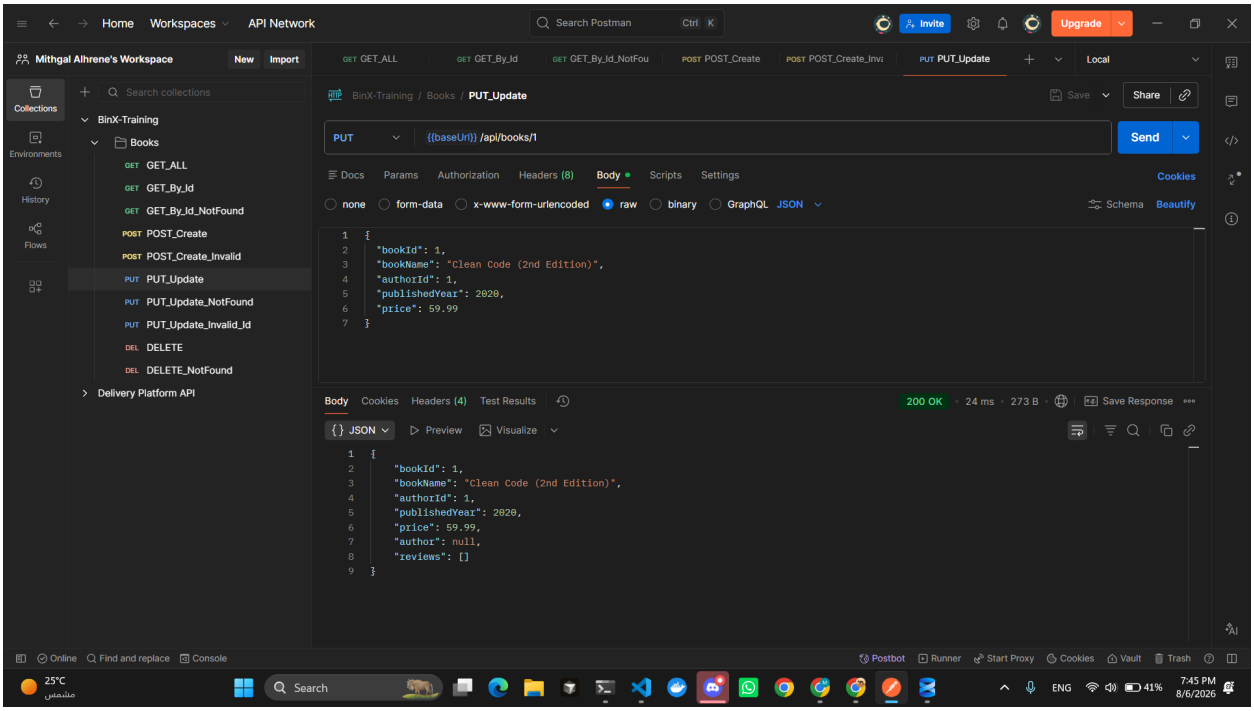
PUT_Update

Method: PUT
URL: {{baseUrl}}/api/books/1
Purpose: Executes the PUT Update operation.

Request Body:

> ***{ "bookId":1,"bookName":"Clean Code (2nd Edition)","authorId":1,"publishedYear":2020,"price":59.99 }***

Success Response: 200 OK



PUT_Update_NotFound

Method: PUT

URL: {{baseUrl}}/api/books/99999

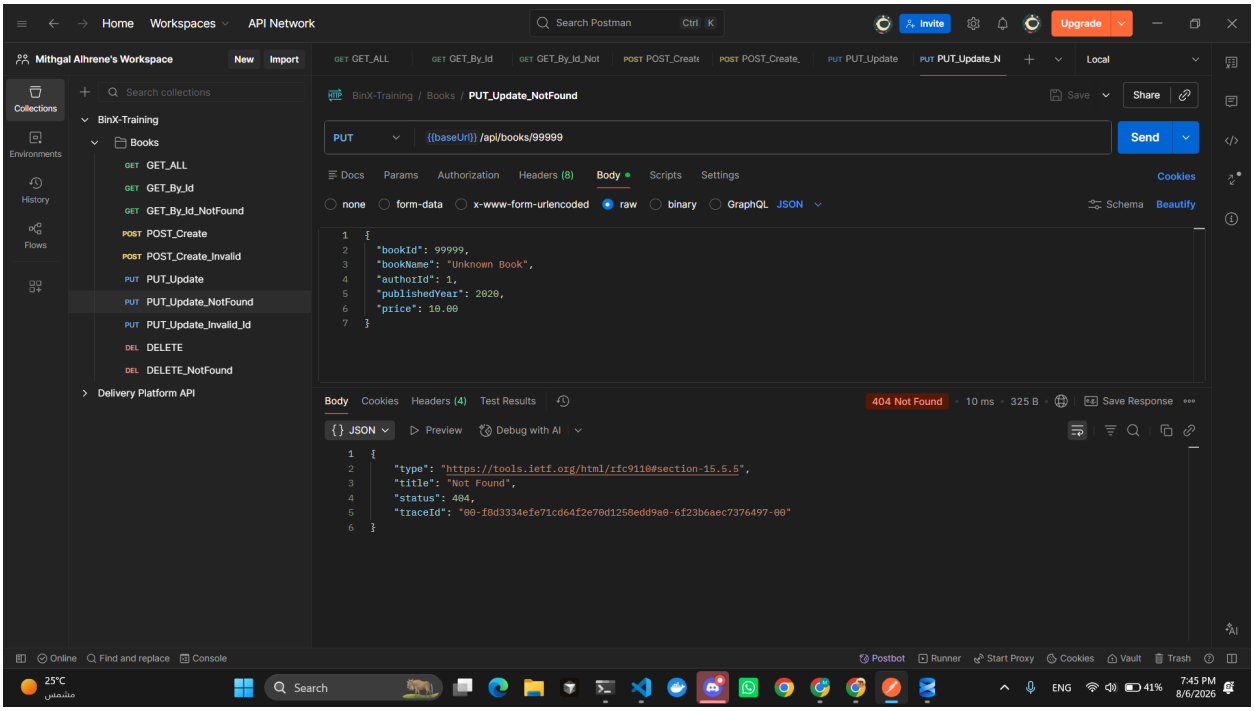
Purpose: Executes the PUT Update NotFound operation.

Request Body:

> ***{ "bookId":99999,"bookName":"Unknown Book","authorId":1,"publishedYear":2020,"price":10.00 }***

Success Response: 404 Not Found

Possible Error Response: 404 Not Found



PUT_Update_Invalid_Id

Method: PUT

URL: {{baseUrl}}/api/books/abc

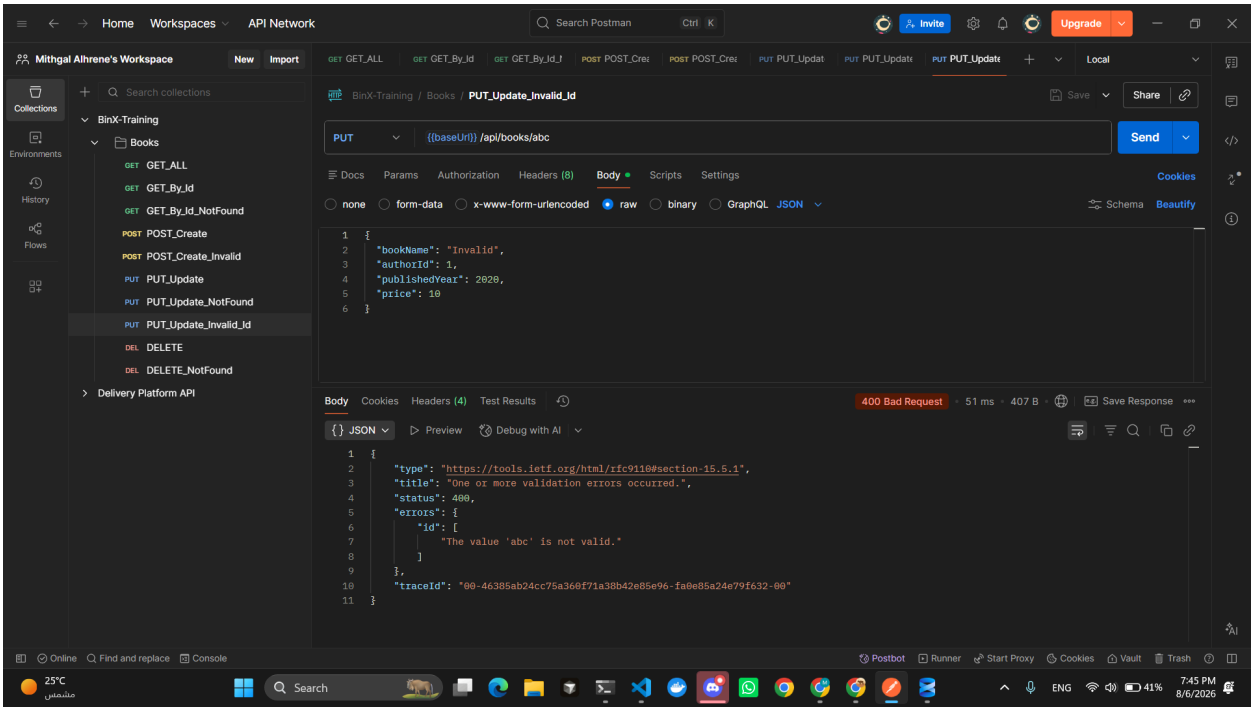
Purpose: Executes the PUT Update Invalid Id operation.

Request Body:

> ***{ "bookName":"Invalid","authorId":1,"publishedYear":2020,"price":10 }***

Success Response: 400 Bad Request

Possible Error Response: 400 Bad Request



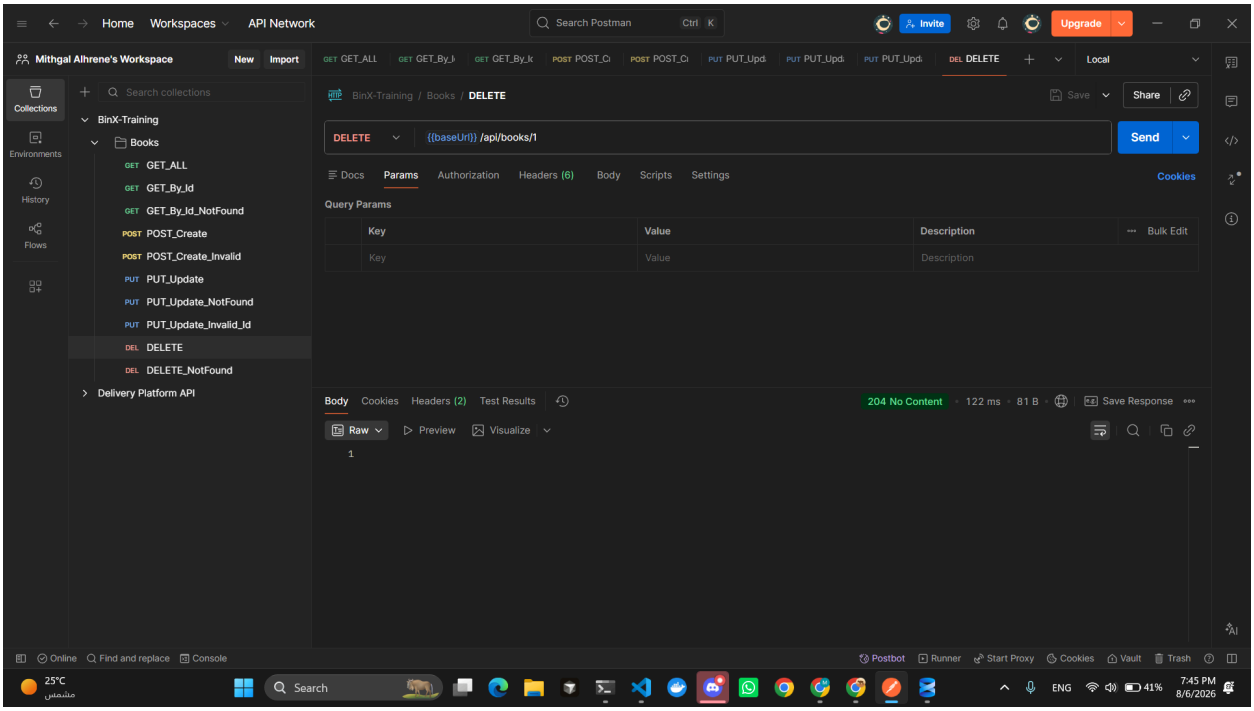
DELETE

Method: DELETE
URL: {{baseUrl}}/api/books/1
Purpose: Executes the DELETE operation.

Request Body:

No request body required.

Success Response: 204 No Content



DELETE_NotFound

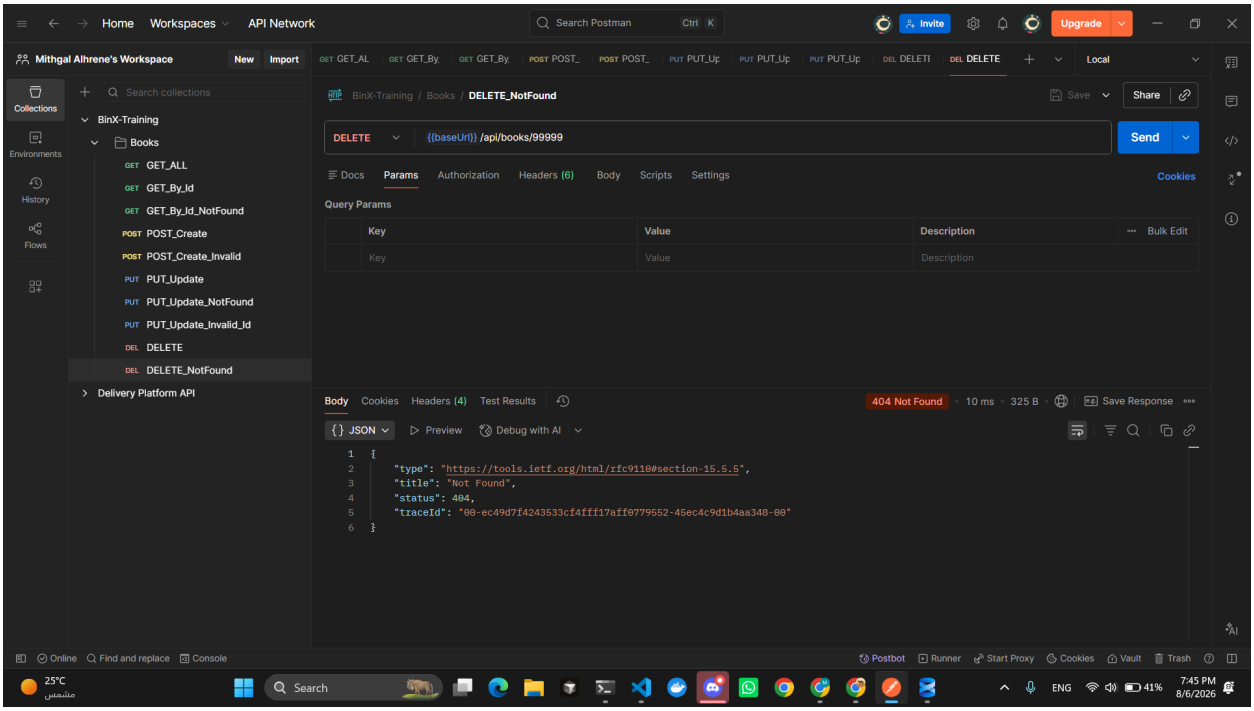
Method: DELETE
URL: {{baseUrl}}/api/books/99999
Purpose: Executes the DELETE NotFound operation.

Request Body:

No request body required.

Success Response: 404 Not Found

Possible Error Response: 404 Not Found



5. Happy Path Testing

Successful execution was verified for GET_ALL (200 OK), GET_By_Id (200 OK), POST_Create (201 Created), PUT_Update (200 OK), and DELETE (204 No Content).

6. Error Path Testing

Negative scenarios verified include GET_By_Id_NotFound (404), PUT_Update_NotFound (404), PUT_Update_Invalid_Id (400), DELETE_NotFound (404), and POST_Create_Invalid (500 Internal Server Error).

7. Postman Tests

The Postman Collection contains test scripts for GET_By_Id (200), GET_By_Id_NotFound (404), and POST_Create (201 with response ID verification), as defined in the provided collection.

8. Conclusion

The provided Postman Collection demonstrates CRUD functionality for the Book Management API. The included screenshots confirm both successful and error scenarios, making this document suitable for internship submission.